Chapter 8

CS1 course package - Hawaii Pacific University (HPU)

Mary L. Smith[1]

[1]HPU University

Abstract

This chapter describes how Hawaii Pacific University (HPU) incorporated Parallel and Distributed Computing (PDC) concepts into sections of an introductory Computer Science I (CS1) course through unplugged and plugged-in learning activities centered on hands-on experiences that emphasized the PDC concepts being introduced. The activities were designed so that students did not need experience writing code in parallel. Instead, students first explored brief explanations of core PDC concepts and then participated in collaborative and interactive activities that allowed them to observe and discuss, the concepts. These experiences reinforced student understanding of topics such as concurrency, message passing, shared memory, scheduling, load balancing, speed up and Amdahl's law while creating an engaging learning environment.

Descriptor Elements

Programming Language Employed: Java

Textbook Used: Starting Out With Java: Control Structures through Objects, 8th edition, Pearson Revel 2022, by Tony Gaddis.

Relevant core courses: The CS1 course includes CSCI 2911(3-credit lecture)/2916(1-credit lab)

Pre-requisites: None

Relevant PDC Topics and Bloom's Levels Per Activity: Table 8.1 summarizes the primary Bloom's Revised Taxonomy levels[1] addressed by each PDC activity. The unplugged activities emphasize the Remembering and Understanding levels, while the plugged-in Penny Sorting activity additionally incorporates Applying through students' review and minor modification of existing Java programs.

Table 8.1: Bloom's Taxonomy Levels Addressed by Each PDC Activity

PDC Concept	**8.3.1 Flag Maker (Unplugged)**	**8.3.2 Penny Sorting (Unplugged)**	**8.4.1 Sorting (Plugged-in)**	**8.6.1 MP Writing (Unplugged)**
Concurrency	RE, UN	RE, UN	UN, AP	RE, UN
Message Passing	RE, UN	RE, UN	UN, AP	RE, UN
Shared Memory	RE, UN	RE, UN	UN, AP	RE, UN
Scheduling / Load Balancing	RE, UN	RE, UN	UN, AP	RE, UN
Speedup / Amdahl's Law	RE, UN	RE, UN	UN, AP	RE, UN
Bloom's Revised Taxonomy Levels: RE-remembering, UN-understanding, AP-applying, AN-analyzing, EV-evaluating, CR-creating				

Activity Learning outcomes: The learning objectives for the activities presented in Sections 8.3.1, 8.3.2, and 8.6.1 primarily align with the Remembering (RE) and Understanding (UN) levels of Bloom's Revised Taxonomy, as shown in Table 8.1. These activities are designed to help students develop foundational knowledge of PDC concepts and build a conceptual understanding of how parallel systems operate. In contrast, the activity presented in Section 8.4.1 extends students' learning by moving beyond conceptual understanding and incorporating learning objectives at the Applying (AP) level of Bloom's Taxonomy. While students did not develop the code independently, they engaged in the Applying level of Bloom's Taxonomy by reviewing, interpreting, and making minor modifications to an existing Java programs, demonstrating how the concepts learned during the unplugged activity could be applied in a programming context.

Overall Learning outcomes: While each PDC activity has its own specific learning objectives, the activities collectively support the following overarching learning outcomes. Upon completion of the integrated PDC activities, students should be able to:

1. **Identify** fundamental Parallel and Distributed Computing (PDC) concepts, including sequential and parallel execution, data partitioning, task decomposition, load balancing, synchronization, communication overhead, speedup, scalability, and Amdahl's Law. *(Remembering)*
2. **Explain** how these foundational PDC concepts influence the design, execution, and performance of parallel programs, including the relationships among workload distribution, processor coordination, and overall execution time. *(Understanding)*
3. **Apply** foundational PDC concepts by interpreting and making minor modifications to Java programs that demonstrate parallel programming constructs, including Java's parallel libraries and the Fork/Join framework. *(Applying)*

Prior to each unplugged activity, brief introductory videos on PDC were shown to provide students with foundational background knowledge and context for the concepts explored during the activities. After Activities 8.3.1,8.3.2, and 8.6.1the class held a post activity discussion to review the key insights gained. Participants compared how tasks were coordinated, analyzed their recorded times, and reflected on the challenges they encountered. The instructor guided the conversation, helping students connect their observations to learning objectives, defining parallel processing, explaining speedup, identifying factors that influence Amdahl's Law, understanding shared memory, and examining how parallel processing and effective task distribution improve efficiency. Through this discussion, participants also came to understand why adding more processors did not always lead to unlimited speedup.

Context for use: HPU is a private, mid-sized institution located in Honolulu, HI with a total enrollment of approximately 4,900 students, including approximately 3,500 undergraduates and 1,400 graduate students. The university is widely recognized for its highly diverse student population, which includes learners from across the United States and more than 60 countries, contributing to a distinctly global campus environment. HPU provides a broad array of undergraduate and graduate programs spanning

business, health sciences, natural sciences, liberal arts, and professional studies. The Department of Computer Science, located within the College of Natural and Computational Sciences, offers both a major and a minor in computer science, with small class sizes typically ranging from 5 to 20 students approximately 100 students currently pursuing the major.

8.1 Introduction

This chapter presents Hawai'i Pacific University's (HPU) integration of foundational parallel and distributed computing (PDC) concepts into an introductory Computer Science I (CS1) course comprising a three-credit lecture and a one-credit laboratory. The chapter describes the design, implementation, and classroom use of a series of unplugged, hands-on learning activities together with a complementary plugged-in programming activity that introduces students to fundamental PDC concepts early in the computing curriculum. The instructional activities are designed to foster conceptual understanding without requiring prior experience in parallel programming. Through active learning, students explore fundamental topics including concurrency, message passing, shared memory, scheduling, load balancing, speedup, and Amdahl's Law. Each activity combines a brief conceptual introduction with collaborative, hands-on exploration, providing students with an engaging and accessible foundation for understanding the principles of parallelism and their importance in modern computing systems.

8.2 How CS1 was changed and PDC topics integrated

There were no significant changes made to the CS1 course; instead of removing existing CS1 topics, the instructor reduced the time previously spent on extended practice with concepts such as classes and class relationships, knowing these ideas would be revisited in greater depth in CS2. The course incorporated PDC concepts alongside the established curriculum rather than replacing or restructuring prior content. These concepts were introduced through a mix of lectures, class discussions, animated videos, and both unplugged and plugged-in lab activities. Typically, one lab session was devoted to these topics, allowing students to engage with the material through guided activities while still maintaining full coverage of the core CS1 content.

8.2.1 Integrated Syllabi (Pre and Post)

The overall structure and progression of the course syllabus remained largely the same before and after the introduction of PDC concepts. In both versions, the course begins with introductory programming content, including Java fundamentals, decision structures, loops and files, methods, text processing, regular expressions, classes, arrays and ArrayLists, testing with JUnit, and concludes with GUI development and a course wrap-up. The course continued to emphasize foundational programming knowledge and object-oriented design principles. The primary change to the revised syllabus was the addition of a dedicated "Parallel & Distributed Computing Fundamentals" unit near the end of the course, after students had developed sufficient experience with classes, arrays, and general problem solving in Java. In the earlier version of the syllabus, PDC concepts were not formally included and may have been referenced only briefly, if at all. Table 8.2 summarizes the differences between the pre-PDC and post-PDC CS1 syllabi, illustrating where PDC concepts and related instructional activities were integrated in the semester.

Table 8.2: Syllabus Pre- and Post-PDC Topics Added

Week	**Pre-PDC Topics Covered**	**Post-PDC Topics Covered**	**PDC Activities**
1	Chapter 1 – Introduction to Class; Introduction to Computers and Java	Chapter 1 – Introduction to Class; Introduction to Computers and Java	
2	Chapter 2 – Java Fundamentals; Decision Structures	Chapter 2 – Java Fundamentals; Decision Structures	
3	Chapters 2, 3 – Decision Structures	Chapters 2, 3 – Decision Structures	
4	Chapters 3, 4 – Decision Structures; Loops & Files	Chapters 3, 4 – Decision Structures; Loops & Files	
5	Chapter 4 – Loops & Files; Methods	Chapter 4 – Loops & Files; Methods	
6	Chapter 9 – Text Processing; Regular Expressions	Chapter 9 – Text Processing; Regular Expressions	
7	Chapter 5 – Methods	Chapter 5 – Methods	
8	Chapter 5 – Methods	Chapter 5 – Methods	
9	Break	Break	
10	Chapter 6 – Classes	Chapter 6 – Classes	
11	Chapter 6 – Classes	Chapter 6 – Classes	
12	Chapters 6, 7 – Classes; Arrays and the ArrayList Class	Chapters 6, 7 – Classes; Arrays and the ArrayList Class	
13	Chapters 7, 8 – Arrays and the ArrayList Class; Second Look at Classes	Chapters 7, 8 – Arrays and the ArrayList Class; Second Look at Classes; Test-Driven Development; JUnit Testing	
14	Chapters 8, 10 – Second Look at Classes, Inheritance	Chapters 8, 10 – Second Look at Classes, Inheritance, Introduction to GUI	

Continued on next page

Table 8.2 (continued)

Week	Pre-PDC Topics Covered	Post-PDC Topics Covered	PDC Activities
15	Introduction to GUI; Test-Driven Development; JUnit Testing	Introduction to PDC Fundamentals	One unplugged activity (Flag Maker, Penny Sorting, or the Multiple Processors (MP) Writing exercise) and one corresponding plugged-in activity (Penny Sorting).
16	Wrapping Things Up	Wrapping Things Up	Completion of the plugged-in activity (if not completed during Week 15).

8.2.2 Highlights

Most of the course content remained unchanged. Core programming topics, the course's overall pacing, and the sequence of foundational Java concepts were preserved to maintain continuity with the existing curriculum and learning objectives. Students still progressed through the same essential programming concepts in the same general order, ensuring that the course continued to prioritize introductory programming competency and object-oriented development skills. Only minimal changes were necessary to incorporate PDC concepts. The main modification was the insertion of the PDC Fundamentals topic into the syllabus. Because PDC had not previously been taught in depth or systematically integrated into earlier units, very little existing material needed to be removed or substantially revised. Rather than deleting content, the update was primarily additive. Minor adjustments to scheduling and topic emphasis were necessary to accommodate the new unit, but the foundational topics themselves remained intact. This approach allowed the course to introduce students to PDC concepts without disrupting the established structure or reducing coverage of essential introductory programming material.

8.2.3 Challenges

Because the integration of PDC concepts relied primarily on unplugged, discussion-based, and hands-on activities, the implementation introduced few logistical or technical challenges. The overall course structure, programming assignments, and technology requirements remained largely unchanged, making it straightforward to incorporate the new instructional content. No specialized software, hardware, parallel programming libraries, or distributed computing environments were required, allowing the material to be integrated without placing additional technical demands on either students or instructors. Although the curriculum included one plugged-in activity, it was designed as a code review exercise rather than a programming assignment. Students examined and discussed existing Java code that demon-

strated how one of the unplugged activities could be implemented using parallel programming techniques. However, they were not required to write or modify any code themselves. The primary challenge was pedagogical rather than technical. As the course serves as an introduction to programming, PDC concepts had to be presented in a manner that was both accessible and appropriate for students with little or no prior computing experience. The unplugged activities were intentionally designed to introduce foundational concepts, including parallel task execution, coordination, synchronization, load balancing, speedup, and distributed problem-solving through collaborative, hands-on learning experiences rather than complex programming exercises. The complementary plugged-in activity reinforced these concepts by allowing students to connect the unplugged experience with an actual implementation, illustrating how the same ideas could be expressed in code without requiring them to develop parallel programs themselves. This instructional approach enabled students to build a conceptual understanding of PDC principles while avoiding the cognitive overload that can accompany advanced implementation details. Logistically, only minor modifications to the course schedule were needed to accommodate the new PDC module and activities. The unplugged and plugged activities aligned well with existing course objectives and complemented traditional programming topics, requiring only minor changes to assignments, assessments, and instructional materials. Consequently, the integration process was smooth and minimally disruptive. Overall, the use of primarily unplugged activities, supported by a single code-review-based plugged-in activity, minimized many of the technical, instructional, and resource-related barriers that often accompany the introduction of PDC concepts into an introductory computer science curriculum while providing students with an accessible introduction to parallel and distributed computing.

8.2.4 Unplugged Activities

An unplugged activity is an interactive, computer-free instructional exercise that uses physical materials and collaboration to illustrate fundamental computing concepts. Prior to incorporating coded examples, instruction focused primarily on unplugged activities to introduce core PDC concepts without the added complexity of programming syntax. These activities were typically preceded by an introductory discussion of PDC, followed by two short videos that provided foundational context. At least one unplugged activity continues to be incorporated into the course. This hands-on instructional approach offered students an engaging way to explore fundamental concepts in parallel computing while reinforcing the advantages and challenges of multi-core processing. Each activity was structured to begin with a uniprocessor, sequential phase, followed by several parallel phases. Completion times for each phase were recorded, compared, and discussed upon conclusion of the activity. Because the activities shared similar instructional objectives and class time was limited, one or at most two activities were generally selected to cover the foundational PDC concepts. The primary unplugged activities used are Flag Maker and Penny Sorting. The Flag Maker activity involved groups of students coloring a designated flag across multiple phases while simulating processor-based task distribution. A second activity involved Penny Sorting, in which students, again acting as processors, worked in groups to identify the oldest penny across three distinct phases. Students demonstrated a high level of engagement throughout all of the unplugged activities, with the penny-sorting activity generating particularly strong

enthusiasm. The activities fostered substantial student collaboration and were characterized by high levels of interaction, participation, and enthusiasm.

8.3 New Unplugged Activities

8.3.1 Unplugged - Flag Maker Activity

Problem Description, Prerequisites, and Learning Outcomes

The Flag Maker activity is an unplugged, hands-on instructional exercise that introduces introductory computer science students to foundational PDC concepts through collaborative, paper-based simulations in which students work together to color a flag using a variety of sequential and parallel execution strategies. Students progress from sequential execution to increasingly parallel execution scenarios, allowing them to develop an intuitive understanding of how multiple processors can work together to solve a problem. Throughout the activity, students explore core PDC concepts, including concurrency, data parallelism, speedup, scalability, load balancing, synchronization, mutual exclusion, and processor coordination. Designed specifically for CS1 students, the activity requires minimal technical prerequisites and no prior knowledge of parallel programming. Instead, students learn through active participation, collaboration, and guided discussion. During the activity, students assume the role of processors and use different parallel strategies to color the flag of Mauritius while coordinating their work to complete the task as efficiently as possible. By comparing multiple execution strategies, students observe how dividing work among processors can improve performance, how coordination influences execution time, and why synchronization is necessary when processors access shared regions. These experiences provide a conceptual foundation for later programming-based PDC activities.

Prerequisites

There are no prerequisites for this activity, making it suitable for students with little or no prior exposure to programming or PDC concepts.

Learning Outcomes

Upon completion of the activity, students should be able to:

- Explain the differences between sequential execution and parallel execution.
- Explain how partitioning data across multiple processors supports parallel processing.
- Describe how distributing work among processors contributes to effective load balancing.
- Identify the communication and synchronization overhead that can occur during parallel execution.

- Explain why the processor that finishes last determines the overall execution time of a parallel program.
- Explain how the activity demonstrates the concepts of speedup, scalability, and Amdahl's Law.

A complete description of the Flag Maker activity, including the activity phases, facilitator instructions, student worksheets, discussion questions, and assessment materials, is provided in Section 1.3.1.

Implementation Details

The Flag Maker activity was implemented during the Fall 2025 semester. Table 8.3 summarizes the estimated time required for each component of the activity, from the introduction and setup through execution, discussion, and optional assessments. These estimates serve as a general guideline and may vary depending on class size, student engagement, and the amount of discussion incorporated.

Table 8.3: Estimated Instructional Time for Unplugged Flag Maker Activity

Component	**Time (minutes)**
Pre-survey	3–5
Introduction and Instructions	5–10
Activity Setup	3–5
Activity Execution	20–25
Discussion and Debrief	6–10
Post-survey/Engagement Survey	3–5
Total Time	**40–60**

The activity is conducted in multiple phases, with each phase introducing a different execution strategy that illustrates a specific PDC concept. Before each phase begins, the facilitator explains the objectives, reviews the rules, answers questions, and allows groups time to organize and assign roles appropriate for that phase. Students are typically organized into teams of five participants (with additional students forming groups of six when necessary). Alternatively, smaller teams of two or three students may be formed initially and later combined for the parallel execution scenarios. Throughout the activity, students act as processors responsible for coloring individual "pixels" on the flag of Mauritius according to the assigned execution strategy. Each group receives four gridded flag templates one for each phase and a set of colored drawing tools. Once all groups are prepared, the facilitator signals the start of the activity, and all teams begin simultaneously. The facilitator records each group's completion time at the end of each phase and publicly displays the results to facilitate discussion and comparison of performance across different execution strategies.

Following each phase, students reflect on their observations and discuss how different approaches to dividing work, coordinating processors, balancing workloads, and synchronizing access to shared regions affected overall performance. These guided discussions reinforce the targeted PDC concepts before students progress to subsequent phases. Detailed implementation instructions, facilitator notes, student handouts, timing recommendations, and debriefing questions are provided in Section 1.3.1.

- **Materials Required** The Flag Maker activity requires only inexpensive classroom materials, making it easy to implement in a variety of instructional settings. Required materials include:
  - Four sheets of gridded paper for each student group (one for each activity phase).
  - Colored drawing implements (e.g., markers, crayons, colored pencils, or bingo daubers) in red, blue, yellow, and green (or additional colors if a different flag is used).
  - A timer, stopwatch, or mobile phone timer.
- **Instructional materiel (Lectures, Videos, Assignments, etc.)** url

Experience and Teaching Tips

The Flag Maker activity has been an effective way to introduce students to foundational PDC concepts, as it allows them to experience parallel execution through a familiar, visual task. Before each phase begins, allow groups a few minutes to discuss their roles and strategy. This planning period encourages students to think about how work should be divided among processors and naturally introduces concepts such as concurrency, scheduling, and data parallelism. Displaying each group's completion time after each phase also provides students with opportunities to compare performance and predict how increasing the number of processors affects overall execution time. Encourage students to reflect on the differences between the four phases rather than simply focusing on which group finishes first. The sequential phase establishes a baseline for comparison, while the two-processor and four-processor phases illustrate how dividing work among processors can improve performance. Comparing the two four-processor approaches (partitioning by stripes versus partitioning by columns) often leads to meaningful discussions about workload distribution, communication overhead, and why different decomposition strategies may produce different results even when the same number of processors is used. The shared materials used throughout the activity provide a natural demonstration of mutual exclusion and synchronization. Because processors share the same sheet of paper and colored drawing implements, students frequently encounter situations in which they must wait for access to a marker or coordinate their movements to avoid interfering with one another. Rather than preventing these interactions, instructors should use them as teachable moments to discuss contention for shared resources, synchronization, and the role of scheduling algorithms such as first-come, first-served (FCFS) in managing resource access. Students quickly recognize that adding processors alone does not eliminate bottlenecks when shared resources must be coordinated. Conclude the activity with a class discussion comparing the recorded execution times from

each phase. Ask students which approach produced the greatest speedup, which strategy resulted in the best load balance, and what factors prevented perfect linear speedup. Encourage them to consider how communication, synchronization, waiting for shared resources, and differences in processor speed affected the overall completion time. These reflections help students connect their hands-on experiences to the underlying PDC concepts and prepare them to understand similar challenges in software-based parallel programming.

Data and Analysis

The evaluation of this activity incorporated both learning and engagement measures. Learning gains were assessed using a one-group pretest-posttest design in which students completed six multiple-choice questions targeting essential parallel computing topics, including sequential and parallel processing, speedup, contention, scalability, and race conditions. Student engagement was assessed separately using a survey adapted from the Assessing Student Perspective of Engagement in Class Tool (ASPECT) [2]. The survey was administered immediately following the activity and the learning assessment. Table 8.4 summarizes student performance on the pre- and post- learning assessments. Students demonstrated improvements on the questions related to task decomposition and contention, while performance on speedup and scalability remained unchanged. Performance declined on the pipelining question, suggesting that this concept may require additional instructional emphasis. Overall, the average score changed from 83.3% on the pre-test to 80.0% on the post-test. Given the small sample size (n = 6), these results are descriptive and are intended to identify tendencies rather than establish statistical significance.

Table 8.4: Pre-test and Post-test Performance by Question

Question	**Concept**	**Pre-test**	**Post-test**	**Change**
Q1	Task Decomposition	83.3%	100.0%	+16.7%
Q2	Speedup	100.0%	100.0%	0.0%
Q3	Contention	33.3%	50.0%	+16.7%
Q4	Scalability	100.0%	100.0%	0.0%
Q5	Pipelining	100.0%	50.0%	−50.0%
Overall	**Average**	**83.3%**	**80.0%**	−3.3%

The student engagement survey was adapted from the ASPECT [2] and modified to align with the learning objectives of the PDC activity. Although the survey was based on the ASPECT instrument, it was not administered in its original form. The survey questions were grouped according to the primary ASPECT categories for analysis, with two additional PDC-specific questions included to evaluate learning outcomes that were related to this instructional intervention. Figure 8.1illustrates the survey questions grouped by category for the Flag Maker activity.

Table 8.5 summarizes the student engagement survey results grouped according to the ASPECT categories, along with two additional Curriculum Integration items included to evaluate the instructional intervention. Overall, students reported highly positive perceptions of the activity, with mean scores ranging from 4.17 to 4.59 on a five-point Likert scale.

Category and Survey Questions
Value of Group Activity*
• Explaining the material to my group improved my understanding of it.
• Having the material explained to me by my group members improved my understanding of it.
• Group discussions during the activity contributed to my understanding of parallel computing.
• I had fun during the activity.
• Overall, the other members of my group made valuable contributions during the activity.
• I would prefer to take a class that includes this group activity over one that does not.
• I am confident in my understanding of the material presented during the activity.
• The activity increased my understanding of parallel computing.
• The activity stimulated my interest in parallel computing.
Personal Effort*
• I made a valuable contribution to my group during this activity.
• I was focused during the activity.
• I worked hard during the activity.
Instructor Contribution*
• The instructor seemed prepared for the activity.
• The instructor put a good deal of effort into my learning from the activity.
• The instructor's enthusiasm made me more interested in the activity.
• The instructor was available to answer questions during the activity.
Curriculum Integration
• The activity increased my understanding of loops.
• I like that the activity tied into the class's current programming topics.

Note. The student engagement survey was adapted from the *Assessing Student Perspective of Engagement in Class Tool* (ASPECT) [1]. Several survey items were modified to reference the PDC activity and its learning objectives, and two PDC-specific questions were added to evaluate outcomes unique to this instructional intervention. The survey retained the three primary ASPECT categories noted by an asterisk—*Value of Group Activity, Personal Effort, and Instructor Contribution*—for analysis.

Figure 8.1: Flag Maker Aspect Categories

The highest ratings were observed for **Instructor Contribution** (M = 4.59), indicating that students perceived the instructor as well prepared, enthusiastic, supportive, and committed to their learning throughout the activity. These results suggest that instructor facilitation contributed positively to the learning experience.

The **Personal Effort** category also received high ratings (M = 4.33), indicating that students remained focused during the activity, worked hard, and believed they made meaningful contributions to their groups. These findings suggest that the collaborative nature of the activity promoted active participation and individual engagement.

Students also responded favorably to the **Value of the Group Activity** (M = 4.30), indicating that they enjoyed the activity, found the collaborative discussions beneficial, and believed the activity increased their understanding of and interest in parallel computing. The positive ratings suggest that students perceived the activity as a valuable addition to the course and appreciated learning through peer interaction.

The **Curriculum Integration** (M = 4.17) also received positive responses. Students agreed that the activity effectively connected with the programming topics covered in the course and contributed to their understanding of loops. Although these items received the lowest mean among the four categories, the ratings remained well above the midpoint of the scale, suggesting that students viewed the activity as relevant to both the course content and their developing programming knowledge.

Overall, the survey results indicate that the activity successfully engaged students, encouraged active participation, and fostered positive perceptions of learning introductory PDC concepts. While the findings should be interpreted cautiously because of the small sample size ($n = 3$), they provide encouraging evidence that collaborative, activity-based instruction can effectively support student engagement in an introductory computer science course. The open-ended survey responses further reinforced these findings. When asked what they found most interesting, students highlighted fundamental PDC concepts, including race conditions, how multiple processors work and the challenges associated with coordinating them, and using multiple processors to solve a problem. These responses suggest that the activity successfully introduced students to core PDC concepts beyond traditional sequential programming. When asked how the activity could be improved, students offered only minor suggestions, such as using colored highlighters to better align with the activity format and providing a slide deck or PDF version of the presentation for students who do not have access to Microsoft PowerPoint. One student indicated that no improvements were necessary. Overall, the qualitative responses complement the quantitative survey results, demonstrating that students found the activity engaging, informative, and well designed while identifying only minor opportunities for enhancement.

Table 8.5: Student Engagement Survey Results Grouped by ASPECT Categories

Category	**Items**	**Mean**
Value of the Group Activity (ASPECT)	9	4.30
Personal Effort (ASPECT)	3	4.33
Instructor Contribution (ASPECT)	4	4.59
Curriculum Integration	2	4.17
Overall	**18**	**4.35**

8.3.2 Unplugged - Penny Sorting

Problem Description, Prerequisites, and Learning Outcomes

The Penny Sorting activity is an unplugged, hands-on instructional exercise that introduces introductory computer science students to foundational PDC concepts through a collaborative sorting-and-searching task. Students assume the role of processors and work individually or in parallel to identify the oldest penny from one or more coin collections. As the activity progresses, students compare sequential execution with multiple parallel execution strategies to observe how workload distribution and processor coordination influence performance. The activity requires minimal technical prerequisites and is intended for

CS1 students with no prior experience in parallel programming. Through active participation, students develop an intuitive understanding of task decomposition, data partitioning, speedup, scalability, communication overhead, and load balancing. The activity also introduces the practical effects of balanced and unbalanced workloads, providing a concrete demonstration of Amdahl's Law and the factors that influence parallel performance.

Prerequisites

There are no prerequisites for this activity, making it suitable for students with little or no prior exposure to programming or PDC concepts.

Learning Outcomes

Upon completion of the activity, students should be able to:

- Explain the differences between sequential execution and parallel execution.
- Explain how partitioning data across multiple processors supports parallel processing.
- Describe how distributing work among processors contributes to effective load balancing.
- Identify the communication and synchronization overhead that can occur during parallel execution.
- Explain why the processor that finishes last determines the overall execution time of a parallel program.
- Explain how the activity demonstrates the concepts of speedup, scalability, and Amdahl's Law.

A complete description of the activity, including facilitator instructions, activity phases, discussion questions, timing recommendations, and assessment materials, is provided in the corresponding activity section 1.3.2.

Implementation Details

The Penny Sorting activity was performed in the Spring 2026 semester Table 8.6 summarizes the estimated time required for each component of the activity, from the introduction and setup through execution, discussion, and optional assessments. These estimates serve as a general guideline and may vary depending on class size, student engagement, and the amount of discussion incorporated.

Students work in groups of approximately five or six participants, although smaller groups may also be used with minor modifications. Throughout the activity, participants act as processors responsible for searching their assigned collection of pennies to identify the oldest coin. The instructor (or teaching assistant) manages task scheduling and distributes pennies for each phase to illustrate different workload distributions. The activity consists of three progressively more complex execution scenarios:

Table 8.6: Estimated Instructional Time for Unplugged Penny Sorting

Component	**Time (minutes)**
Pre-survey	3–5
Introduction and Instructions	5–10
Activity Setup	3–5
Activity Execution	25–30
Discussion and Debrief	6–10
Post-survey/Engagement Survey	3–5
Total Time	**45–65**

Phase 1 – Sequential (Uniprocessor) Execution: One participant performs the search, another records the execution time, and the remaining group members observe the process. This phase establishes a sequential performance baseline.

Phase 2 – Balanced Two-Processor Parallel Execution: The pennies are evenly divided between two participants, resulting in a load-balanced parallel execution. Both participants search simultaneously, communicate their results, and determine the overall oldest penny while another participant records the total execution time. This phase demonstrates the benefits of parallel execution and the communication required between processors.

Phase 3 – Load-Imbalanced Five-Processor Parallel Execution: The pennies are distributed unevenly among five participants, with one participant receiving substantially more pennies than the others. This intentionally creates a load imbalance to illustrate how uneven workload distribution limits parallel performance and to demonstrate the practical implications of Amdahl's Law. An additional participant records the overall execution time while the processors communicate to determine the oldest penny across all bags.

Following each phase, students compare execution times, discuss the impact of workload distribution and communication overhead, and relate their observations to fundamental PDC concepts. Complete implementation instructions are provided in Section 1.3.2.

- **Materials Required** The Penny Sorting activity uses readily available materials, with pennies serving as the primary instructional resource. The class size and the number of participating groups determine the number of pennies required. In situations where sufficient pennies are unavailable, such as after the discontinuation of penny production, paper replicas with different mint years may be used as an effective substitute without altering the learning objectives or the implementation of the activity. Required materials include:
  - One bag of pennies for each group (approximately 50 pennies, or another even number appropriate for the class size) for Phases 1 and 3.
  - Two bags of pennies for each group, together containing the same total number of pennies used in Phase 1, for the balanced parallel execution phase.
  - A timer, stopwatch, or mobile phone timer for each group.

  - *Optional:* A Penny Sorting worksheet for recording sorting times, search times, and observations during each phase.

- **Instructional Materiel (Lectures, Videos, Assignments, etc.)** `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter8-HPU`

Experience and Teaching Tips

The Penny Sorting activity provides an engaging and intuitive introduction to data parallelism, scheduling, load balancing, speedup, and Amdahl's Law by allowing students to experience how data can be physically partitioned among processors. Before each phase begins, give groups a few minutes to discuss how the work will be organized and ensure that each participant clearly understands their role. Although the instructor assigns the data distribution for each phase, encouraging students to predict which phase will finish fastest and explain why promotes active engagement and creates a foundation for the post-activity discussion. One of the greatest strengths of this activity is the contrast between the three execution scenarios. The sequential phase establishes a baseline for measuring performance, while the balanced two-processor phase demonstrates how evenly distributing data can reduce execution time and improve speedup. In contrast, the five-processor phase intentionally introduces a load imbalance by assigning one participant significantly more pennies than the others. Students quickly observe that the overall completion time depends on the slowest processor rather than the average completion time, providing a concrete illustration of why balanced workloads are critical for efficient parallel execution. Encourage students to pay attention not only to the sorting process but also to the communication required after each participant identifies the oldest penny in their assigned data. The time required to compare results and determine the oldest penny provides an excellent opportunity to discuss communication overhead and why additional processors do not always yield proportional performance improvements. This naturally leads into discussions of speedup and Amdahl's Law, as students recognize that portions of the task, such as combining results, remain sequential and limit the maximum achievable performance gain. Conclude the activity by comparing the recorded execution times from all three phases and asking students to explain the observed differences. Guide the discussion toward concepts such as balanced versus unbalanced data partitioning, scheduling decisions, communication overhead, and scalability. Encourage students to consider how they might redesign the workload in the five-processor phase to achieve better load balancing and greater speedup. These reflections help students connect their observations to fundamental PDC principles and reinforce that effective parallel computing depends not only on increasing the number of processors but also on distributing work efficiently and minimizing coordination overhead.

Data and Analysis

Following the unplugged Penny Sorting activity and the subsequent plugged-in Penny Sorting activity, students completed a single engagement survey to evaluate their perceptions of the overall learning experience. A single post-activity survey was intentionally administered to minimize survey fatigue and reduce the burden on students. As a result, because

the survey was completed only after both instructional activities had been finished, students' responses likely reflected their combined experiences with the unplugged and plugged-in activities rather than their perceptions of either activity individually.

The student engagement survey was adapted from the ASPECT [2] and modified to align with the learning objectives of the PDC activity. Although the survey was based on the ASPECT instrument, it was not administered in its original form. The survey consisted of quantitative and qualitative items. The quantitative questions were rated using a five-point Likert scale, while the qualitative questions solicited open-ended feedback from students. For analysis, the quantitative survey questions were grouped according to the primary ASPECT categories. In addition, one question assessed students' prior knowledge of PDC, and one question asked students to rate the extent to which the plugged-in activity helped them understand PDC concepts. Two open-ended questions asked students to provide suggestions for improving the unplugged activity and the plugged-in activity. Figure 9.2 illustrates the survey questions grouped by category for the unplugged and plugged-in Penny Sorting activities.

Table 8.7 summarizes the quantitative survey results grouped according to the adapted ASPECT categories. Because the activity was implemented in a small Computer Science I course, the survey responses represent a limited sample of six students (n = 6). Consequently, the results should be interpreted as descriptive of this cohort rather than generalized to a broader student population. Students reported very limited prior knowledge of PDC before participating in the activities (M = 1.00), indicating that the concepts introduced were largely new to the class. Among the adapted ASPECT categories, **Instructor Contribution** received the highest mean score (M = 4.67), suggesting that students viewed the instructor as well prepared, enthusiastic, and readily available to answer questions throughout the activities. The **Personal Effort** category also received a high rating (m = 4.44), indicating that students remained focused, worked diligently, and believed they made meaningful contributions to their groups during the activity. Students rated the **Value of the Group Activity** positively (M = 4.22), suggesting that the collaborative nature of the Penny Sorting activity supported their understanding of foundational parallel computing concepts. Students generally agreed that explaining concepts to peers, receiving explanations from group members, participating in group discussions, and working collaboratively contributed to their learning while making the activity engaging and enjoyable. The plugged-in activity also received favorable ratings (M = 4.00), indicating that students believed it reinforced the PDC concepts introduced during the unplugged exercise. Although this category received the lowest mean among the post-activity categories, the rating remained positive, suggesting that the programming activity effectively complemented the conceptual understanding developed during the unplugged activity. Student comments, however, indicate that additional time spent reviewing or writing the code could further strengthen this component of the lesson. While the small sample size limits broad conclusions, the findings suggest that combining an unplugged collaborative activity with a subsequent programming exercise can provide an engaging and effective introduction to foundational Parallel and Distributed Computing concepts for students with little or no prior PDC experience. Analysis of the open-ended responses revealed several recurring themes. Students appreciated the collaborative structure of the unplugged activity and the opportunity to communicate and coordinate with their peers while completing the task. Several students suggested incorpo-

Category and Survey Questions
Prior Knowledge
• My knowledge of Parallel and Distributed Computing (PDC) before the activities.
Value of Group Activity*
• Explaining the material to my group improved my understanding of it.
• Having the material explained to me by my group members improved my understanding of the material.
• Group discussion during the Penny activity contributed to my understanding of parallel computing fundamentals.
• I had fun during today's Penny activity.
• Overall, the other members of my group made valuable contributions during the Penny activity.
• I would prefer to take a class that includes this group activity over one that does not include this Penny group activity.
• I am confident in my understanding of the material presented during today's Penny activity.
• The Penny activity increased my understanding of the fundamentals of parallel computing.
• The Penny activity stimulated my interest in parallel computing.
Personal Effort*
• I made a valuable contribution to my group today.
• I was focused during today's Penny activity.
• I worked hard during today's Penny activity.
Instructor Contribution*
• The instructor's enthusiasm made me more interested in the Penny activity.
• The instructor put a good deal of effort into my learning for today's class.
• The instructor seemed prepared for the Penny activity.
• The instructor and/or teaching assistants were available to answer questions during the Penny activity.
Programming Activity Effectiveness
• The plugged-in coding activity helped me understand PDC concepts.
Open-Ended Feedback
• How could the unplugged activity be improved?
• How could the plugged-in coding activity be improved?

Note. The student engagement survey was adapted from the *Assessing Student Perspective of Engagement in Class Tool* (ASPECT) [1]. The original ASPECT categories—*Value of Group Activity*, *Personal Effort*, and *Instructor Contribution*—were retained to evaluate student engagement following the unplugged and plugged-in Penny Sorting activities. Several survey items were modified to reference the PDC activities and their learning objectives. The three primary ASPECT categories, identified by an asterisk, were retained for analysis. Additional items assessed students' prior knowledge of PDC, the effectiveness of the subsequent plugged-in coding activity, and qualitative feedback regarding both instructional activities. Because the survey was administered after students completed both activities, responses may reflect students' perceptions of the overall learning experience rather than either activity individually.

Figure 8.2: Penny Sorting Aspect Categories

rating additional practice or trial runs before beginning the activity, while others noted that a larger class size would provide greater opportunities for interaction. Feedback regarding the plugged activity primarily focused on increasing the amount of instructional time devoted to the programming exercise, allowing students to write the parallel code themselves, providing a more detailed walkthrough of the implementation, and incorporating additional visual demonstrations. One student also noted that an instructional video failed to load during the activity. Overall, the qualitative feedback complements the quantitative findings by suggesting that students valued both instructional components while identifying opportunities to strengthen the programming portion of the lesson further.

Table 8.7: Student Engagement Survey Results Grouped by Adapted ASPECT Categories for the Penny Sorting Activity

Category	**Items**	**Mean**
Prior Knowledge	1	1.00
Value of the Group Activity (ASPECT)	9	4.22
Personal Effort (ASPECT)	3	4.44
Instructor Contribution (ASPECT)	4	4.67
Plugged Activity	1	4.00
Overall	**18**	**4.17**

8.4 New Plugged-in Activities

8.4.1 Plugged-in - Parallel Sort Activity

Problem Description, Prerequisites, and Learning Outcomes

The plugged Penny Sorting activity builds on the unplugged Penny Sorting exercise by introducing students to the programmatic implementation of sequential and parallel sorting in Java. The activity is designed to help students connect the parallel computing concepts experienced during the hands-on activity with real-world programming techniques. Because many undergraduate students have little or no prior exposure to parallel programming, the activity provides an accessible introduction to how modern programming languages and libraries support parallel execution. Students examine and compare sequential and parallel implementations while exploring concepts such as task decomposition, concurrency, synchronization, load balancing, scalability, and speedup. Two Java programs support the activity. The first establishes a sequential baseline for comparison, while the second demonstrates multiple approaches to parallel programming using Java's built-in parallel libraries and the Fork/Join framework. By comparing these implementations, students gain an understanding of the different levels of abstraction available for parallel programming and the trade-offs between implementation simplicity, programmer control, and performance. The activity reinforces the relationship between the unplugged exercise and practical software development by illustrating how the same parallel-computing concepts are applied in real programs.

Prerequisites

Students should have:

- Completed the unplugged Penny Sorting activity.
- A basic understanding of Java programming, including variables, loops, methods, arrays, and classes.
- An understanding of sequential program execution.
- Experience reading and interpreting Java source code.

- No prior experience with parallel programming is required.

Learning Outcomes

After completing this activity, students should be able to:

- Explain the differences between sequential and parallel execution.
- Describe the principles of data, functional, and task parallelism.
- Explain how Java supports parallel programming through built-in libraries and the Fork/Join framework.
- Describe the roles of task decomposition, concurrency, synchronization, load balancing, communication overhead, and speedup in parallel applications.
- Explain how different parallel programming approaches vary in implementation complexity, programmer control, scalability, and performance.
- Explain the connection between the Penny Sorting activity and its corresponding implementation in Java programs.

Implementation Details

Table 8.8 summarizes the estimated time required for each component of the activity, from the introduction and setup through execution, discussion, and optional assessments. These estimates serve as a general guideline and may vary depending on class size, student engagement, and the amount of discussion incorporated.

Table 8.8: Estimated Instructional Time for Plugged-in Penny Sorting

Component	**Time (minutes)**
Pre-survey	3–5
Introduction and Instructions	2–5
Activity Setup	2–5
Activity Execution	15–20
Discussion and Debrief	5–10
Post-survey/Engagement Survey	3–5
Total Time	**30–50**

- **Materials Required** The plugged programming activity requires minimal instructional resources and is designed to be delivered primarily as an instructor-led demonstration. The following materials are required to implement the activity:

  - An instructor computer with the Java Development Kit (JDK) and a Java Integrated Development Environment (IDE), such as IntelliJ IDEA, Eclipse, or NetBeans.
  - The provided sequential and parallel Java programs used to demonstrate the PDC concepts.
  - A classroom projector or large display for presenting the source code, program execution, and results to students.
  - *Optional:* Presentation slides to introduce the concepts and summarize the results.
  - *Optional:* Student handouts or worksheets for recording observations and comparing the sequential and parallel implementations.

- **Instructional Material (Lectures, Videos, Assignments, Tests)** `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter8-HPU`

- **How-To Implement** After students complete the unplugged Penny Sorting activity and have been introduced to arrays, the instructor transitions to a guided code walkthrough that demonstrates how the same concepts can be implemented programmatically in Java. Because students typically have little or no prior experience with parallel programming, the activity begins with a sequential implementation before introducing three progressively more advanced parallel approaches. Throughout the walkthrough, students examine the source code, discuss how each implementation performs the sorting task, identify where parallelism occurs, and compare the advantages and trade-offs of each approach. The instructor encourages students to predict which implementation will perform best before reviewing execution times and discussing the results.

  The activity begins with PennySortingSequential.java, which generates an array of one million randomly selected years (1900–2026) and sorts the array using a traditional sequential algorithm. Students first examine how a single processor performs every comparison and sorting operation one step at a time, establishing a baseline for understanding execution time and the limitations of sequential processing. During the discussion, students identify how many processors are performing the work, consider which portions of the algorithm could potentially execute simultaneously, and discuss the limitations of relying on a single processor.

  Students then transition to PennySortingParallel.java, which demonstrates three different approaches to parallel programming while sorting identical copies of the same one-million-element dataset. The first implementation introduces data parallelism using Java's built-in Arrays.parallelSort() method. Students examine how the Java runtime automatically partitions the array into smaller sections, sorts each section concurrently using the Fork/Join framework, and merges the results without requiring the programmer to explicitly manage threads or synchronization. The discussion focuses on data decomposition, automatic load balancing, multicore execution, and how built-in libraries simplify parallel programming while often providing excellent performance.

  Next, students examine functional parallelism using the Java Stream API. The instructor explains how the array is converted into a parallel stream and processed through a

pipeline of operations, allowing the programmer to specify what should be done while Java determines how the work is distributed across multiple threads. This provides an opportunity to introduce declarative programming while discussing why parallel streams may not always outperform Arrays.parallelSort(), including the overhead associated with stream creation, thread coordination, and stateful operations such as sorting.

Finally, students analyze a custom Fork/Join implementation of parallel merge sort, representing task parallelism. Unlike the previous approaches, this implementation requires the programmer to explicitly divide the problem into recursive subtasks, execute those tasks concurrently using invokeAll(), and merge the results. As students trace the recursive execution of the algorithm, they discuss recursive task decomposition, synchronization during merging, selecting appropriate thresholds, and balancing task overhead with potential performance gains. This implementation demonstrates how greater programmer control provides increased flexibility while also increasing implementation complexity.

The activity concludes with a comparison of all four implementations: sequential sorting, Arrays.parallelSort(), parallel streams, and the custom Fork/Join merge sort. Students compare execution times, implementation complexity, scalability, programmer effort, and flexibility while discussing the advantages and disadvantages of each approach. Finally, the instructor guides students in relating these programming implementations back to the unplugged Penny Sorting activity, reinforcing how concepts such as task decomposition, concurrency, synchronization, communication, load balancing, and speedup are represented in both the physical activity and real parallel programs. This progression from unplugged activity to sequential code and increasingly sophisticated parallel implementations helps students connect conceptual understanding with practical parallel programming techniques.

Experience and Teaching Tips

The transition from the unplugged Penny Sorting activity to the Java implementation provided students with an opportunity to connect their conceptual understanding of PDC with practical programming techniques. Before introducing the code, it was helpful to revisit the unplugged activity and ask students to describe how the work had been divided among participants, how communication occurred, and what factors influenced the overall execution time. This discussion allowed students to recognize that the Java programs implemented the same ideas they had already experienced in the hands-on activity.

Because the students had little or no prior exposure to parallel programming, reviewing the code line by line was more effective than expecting students to immediately understand the complete programs independently. Comparing the sequential implementation with the three parallel implementations allowed students to observe that there are multiple ways to express parallelism in Java, each with different levels of abstraction, complexity, and programmer control. Emphasizing the similarities between the sequential and parallel versions before discussing their differences helped reduce student anxiety and kept the focus on the underlying PDC concepts rather than language syntax.

One lesson learned from the initial implementation was that combining both the unplugged and plugged activities into a single class session created a substantial cognitive load for students. Although students found the coding examples valuable, many indicated that they would have benefited from additional time to review the programs, step through the execution, and modify portions of the code themselves. Future offerings would likely benefit from presenting the coding activity during a separate class session after students have had time to reflect on the unplugged exercise. This approach would provide more opportunities for guided practice, discussion, and experimentation while reinforcing the connection between conceptual models and practical parallel programming.

Finally, instructors should encourage students to analyze the benchmark results rather than focusing solely on execution times. The discussion should emphasize why multiple executions are averaged, how JVM warm-up and system variability affect performance measurements, and why increased parallelism does not always result in proportional speedup. Connecting these observations back to the unplugged Penny Sorting activity reinforces that the same principles of data partitioning, load balancing, communication overhead, and scalability apply whether the processors are students sorting pennies or threads executing code.

Data and Analysis

As discussed in Section 8.3.2, students completed a single engagement survey after participating in both the unplugged and plugged-in activities. Because the survey was administered only after both instructional components had been completed, students' responses likely reflected their overall perceptions of the combined learning experience rather than either activity in isolation. For this reason, the comprehensive analysis of the engagement survey is presented in the unplugged Penny Sorting activity section. The survey included one item specifically evaluating the plugged-in activity. Students generally agreed that the coding exercise helped them understand PDC concepts ($M = 4.00$). Although this was the lowest mean among the post-activity quantitative measures, the rating remained positive, suggesting that the programming exercise effectively reinforced the concepts introduced during the unplugged Penny Sorting activity. Qualitative feedback further supported this finding while identifying opportunities for improvement. Several students recommended allocating additional class time for the programming exercise, providing more detailed walkthroughs of the implementation, and allowing students to write portions of the parallel code themselves. One student also suggested incorporating additional visual demonstrations to complement the coding examples. Given the small sample size ($n = 6$), these findings should be interpreted as descriptive of this cohort rather than generalized to a broader population. Future studies involving larger cohorts, multiple course offerings, and separate evaluations of the unplugged and plugged-in activities would help validate these findings and provide a more detailed assessment of the instructional effectiveness of each component individually as well as the combined instructional approach.

8.5 Course-wide Pre and Post Course Surveys

The pre- and post-course surveys collected demographic information, asked students to self-assess their overall computer experience, and measured their perceived experience with various programming languages and programming concepts. Students rated each experience item on a five-point Likert scale, where 1 indicated no experience and 5 indicated extensive experience. For this analysis, the focus is on students' self-reported experience with Java, the programming language used in the course, along with the following programming topics: Data Types, Arithmetic Expressions and Operators, Branching, Loops, Calling Functions/Methods, Arrays, Writing Static Functions, Writing Methods, Creating Objects, Parallel Processing, Distributed Computing, and Event Handling. Comparing students' pre- and post-survey responses provides insight into changes in their perceived knowledge and confidence throughout the course. The comparison tables presented in this section report the mean pre- and post-survey ratings for each programming topic, along with the corresponding mean change between the two surveys. This section examines the survey results across three course offerings. The Fall 2024 offering served as a baseline because no PDC instructional activities were incorporated into the course. In contrast, the Fall 2025 and Summer 2026 offerings integrated PDC activities into the curriculum, allowing changes in students' confidence in both traditional programming topics and introductory PDC concepts to be examined.

The Fall 2024 pre- and post-course surveys were used to evaluate changes in students' self-reported programming knowledge and confidence over the semester. Five students completed both surveys and were included in the comparison. Table 8.9 summarizes students' self-reported confidence levels before and after the course across thirteen programming topics. Overall, students reported increased confidence in most programming concepts following instruction. The largest gains were observed in Writing Methods and Creating Objects, each increasing by 2.0 points on the five-point Likert scale. Significant improvements were also seen in Writing Static Functions (+1.8), Calling Functions/Methods (+1.2), Arrays (+1.2), and Arithmetic Expressions and Operators (+1.0), suggesting that students developed greater confidence in core programming skills throughout the course. Moderate increases were observed in Java, Branching, and Loops (each +0.8), while Data Types increased by 0.4 points. Confidence in Parallel Processing increased slightly from 1.2 to 1.8, indicating a modest improvement despite students having relatively limited exposure to the topic. Distributed Computing showed no overall change, and Event Handling increased only slightly (+0.2). It is important to note that no specific PDC instructional activities were incorporated into this course offering. Consequently, students' familiarity with PDC concepts remained comparatively low, as expected. Given the small sample size (n = 5), these results are intended to provide descriptive insights rather than support statistical inference. Overall, the findings indicate that students perceived meaningful growth in traditional programming knowledge and object-oriented programming skills during the course. At the same time, confidence in PDC-related topics remained limited in the absence of dedicated PDC instruction

The Fall 2025 and Spring 2026 CS1 courses included specific PDC activities. The Fall 2025 pre- and post-course surveys were used to assess changes in students' self-reported pro-

Table 8.9: Comparison of Pre- and Post-Course Survey Results (Fall 2024)

Experience Category	**Pre**	**Post**	**Change**
Java	**2.40**	**3.20**	**+0.80**
Data Types	3.40	3.80	+0.40
Arithmetic Expressions	3.00	4.00	+1.00
Branching	3.00	3.80	+0.80
Loops	3.00	3.80	+0.80
Calling Functions/Methods	3.00	4.20	+1.20
Arrays	2.60	3.60	+1.00
Writing Static Functions	2.00	4.00	+2.00
Writing Methods	2.20	4.20	+2.00
Creating Objects	2.60	4.20	+1.60
Parallel Processing	1.20	1.80	+0.60
Distributed Computing	1.80	1.80	0.00
Event Handling	2.00	2.20	+0.20

gramming experience and confidence throughout the course. Four students completed both surveys and were included in the comparison. Table 8.10 ,summarizes students' self-reported confidence before and after instruction across thirteen programming topics. Overall, students reported increased confidence in all topics after the course. The largest improvement was observed in Java, with the mean self-rating increasing from 1.50 to 3.50 (+2.00). Other substantial gains were found in Writing Static Functions (+1.75), Arrays (+1.50), Parallel Processing (+1.50), and Event Handling (+1.50). Moderate improvements were observed in Writing Methods, Creating Objects, and Distributed Computing (each +1.25), while Calling Functions/Methods increased by 1.00 point. Smaller increases were seen in Data Types, Branching, and Loops (+0.75), and Arithmetic Expressions and Operators showed the smallest improvement (+0.50). Although students' confidence in Parallel Processing and Distributed Computing remained lower than that for many traditional programming topics, both areas demonstrated meaningful increases following instruction, suggesting that the course improved students' familiarity with foundational PDC concepts. Because the sample size was small (n = 4), these results should be interpreted descriptively rather than inferentially. Nevertheless, the overall trend indicates increased student confidence across both traditional programming topics and introductory PDC concepts following the instructional activities.

Table 8.11 summarizes students' self-reported confidence before and after instruction across thirteen programming topics. Overall, students reported substantial increases in confidence across every topic following the course. The largest improvement was observed in Writing Static Functions, with the mean self-rating increasing from 1.00 to 4.00 (+3.00). Other notable gains were found in Writing Methods (+2.80), as well as Java, Calling Functions/Methods, and Arrays, each increasing by 2.60 points. Students also demonstrated great improvements in Branching and Creating Objects, each increasing by 2.40 points. Moderate gains were observed in Data Types and Loops (+1.80), while Arithmetic Expressions and Operators increased by 1.60 points. Confidence in Parallel Processing, Distributed

Table 8.10: Comparison of Pre- and Post-Course Survey Results (Fall 2025)

Experience Category	**Pre**	**Post**	**Change**
Java	**1.50**	**3.50**	**+2.00**
Data Types	3.25	4.00	+0.75
Arithmetic Expressions & Operators	3.25	3.75	+0.50
Branching	3.00	3.75	+0.75
Loops	2.75	3.50	+0.75
Calling Functions/Methods	2.75	3.75	+1.00
Arrays	2.00	3.50	+1.50
Writing Static Functions	1.75	3.50	+1.75
Writing Methods	2.00	3.25	+1.25
Creating Objects	2.25	3.50	+1.25
Parallel Processing	1.50	3.00	+1.50
Distributed Computing	1.75	3.00	+1.25
Event Handling	1.75	3.25	+1.50

Computing, and Event Handling each increased by 1.00 point. Although these PDC-related topics showed smaller gains than many of the traditional programming topics, students' post-survey ratings indicate increased familiarity following their initial exposure to Parallel and Distributed Computing concepts during the course. Although students' post-survey ratings for these topics remained lower than those for traditional programming concepts, the observed increases indicate growing confidence following their initial exposure to PDC. Given the small sample size (n = 4), these findings should be interpreted descriptively rather than inferentially. Nevertheless, the overall pattern of results suggests that students perceived meaningful improvements in both traditional programming skills and introductory PDC concepts throughout the instructional activities.

The largest improvements occurred in writing static functions, writing methods, arrays, calling functions, writing classes, and creating objects, indicating substantial growth in students' confidence with object-oriented programming and software development concepts. Students also reported increased familiarity with topics beyond traditional programming. Average self-reported experience with parallel processing increased from 1.80 to 2.80, distributed computing increased from 1.80 to 2.80, and event handling increased from 2.00 to 3.00. Although these concepts were introduced at an introductory level, the increases suggest that students completed the course with greater awareness and confidence in these advanced computing topics.

Course-wide Pre and Post Course Surveys Overall Conclusions

Across the three course offerings, students consistently reported increased confidence in traditional programming topics, including Java, data types, arithmetic expressions, branching, loops, arrays, calling functions and methods, writing static functions, writing methods, and creating objects. The largest gains were consistently observed in topics related to procedural and object-oriented programming, particularly writing functions and methods, ma-

Table 8.11: Comparison of Pre- and Post-Course Survey Results (Spring 2026)

Experience Category	**Pre**	**Post**	**Change**
Java	**1.20**	**3.80**	**+2.60**
Data Types	2.20	4.00	+1.80
Arithmetic Expressions & Operators	2.40	4.00	+1.60
Branching	1.60	4.00	+2.40
Loops	2.20	4.00	+1.80
Calling Functions/Methods	**1.40**	**4.00**	**+2.60**
Arrays	**1.20**	**3.80**	**+2.60**
Writing Static Functions	**1.00**	**4.00**	**+3.00**
Writing Methods	**1.20**	**4.00**	**+2.80**
Creating Objects	1.60	4.00	+2.40
Parallel Processing	1.80	2.80	+1.00
Distributed Computing	1.80	2.80	+1.00
Event Handling	2.00	3.00	+1.00

nipulating arrays, and creating objects. These are fundamental CS1 learning outcomes that provide the foundation for later coursework in software development and computer science. The observed improvements suggest that students perceived meaningful growth in both their programming knowledge and their ability to apply core programming concepts throughout the semester. An important observation across the three course offerings is that the introduction of PDC activities did not appear to detract from students' perceived learning of the traditional CS1 programming topics. Students in the Fall 2025 and Spring 2026 offerings, which incorporated PDC activities, reported improvements across the same foundational programming concepts as students in the baseline Fall 2024 offering. In many cases, the gains in Java programming, arrays, functions, methods, and object-oriented programming concepts were comparable to or greater than those observed in the course offering that did not include PDC activities. These findings indicate that incorporating PDC instruction did not come at the expense of the core CS1 curriculum. Instead, students continued to report substantial gains in the traditional programming concepts while also developing familiarity with introductory PDC topics. Comparing the baseline course offering (Fall 2024) with the two offerings that incorporated PDC activities reveals a notable difference in students' self-reported confidence related to Parallel Processing and Distributed Computing. During the Fall 2024 offering, when no dedicated PDC instructional activities were included, students reported only modest improvement in Parallel Processing, no improvement in Distributed Computing, and minimal gains in Event Handling. In contrast, students in both the Fall 2025 and Spring 2026 offerings reported larger increases in confidence for Parallel Processing and Distributed Computing after participating in the unplugged and plugged PDC activities. Although post-course ratings for these topics remained lower than those for traditional programming concepts, the observed improvements suggest that students completed the courses with greater familiarity and confidence in these topics than they reported at the beginning of the semester. The Spring 2026 course exhibited the largest overall increases in self-reported confidence across most programming topics. Students reported substantial gains in Java,

branching, arrays, writing static functions, writing methods, creating objects, and calling functions, while also demonstrating positive increases in Parallel Processing and Distributed Computing. The progression of results across the three course offerings suggests that students can develop confidence in advanced computing topics while simultaneously strengthening their understanding of fundamental programming concepts introduced throughout the CS1 curriculum. Student engagement survey responses complemented these findings. Students generally agreed that the programming activity reinforced the concepts introduced during the unplugged activity and helped them better understand parallel computing. Qualitative feedback indicated that students appreciated the connection between the physical activities and their software implementations. Students also suggested several improvements, including allocating additional time for coding, incorporating more instructor-led demonstrations, allowing students to develop portions of the code themselves, and providing additional visual explanations of the algorithms. These recommendations provide valuable guidance for refining future implementations while maintaining the connection between conceptual understanding and practical programming. Because each course offering involved relatively small class sizes (Fall 2024: n = 5; Fall 2025: n = 4; Spring 2026: n = 5 for the course survey and n = 6 for the engagement survey), the findings should be interpreted as descriptive rather than inferential. Although statistical conclusions cannot be drawn from these data, the consistent patterns observed across multiple course offerings provide encouraging preliminary evidence regarding students' perceived growth in traditional programming knowledge and their increased familiarity with foundational Parallel and Distributed Computing concepts following the instructional activities. Future studies involving larger cohorts, multiple institutions, and longitudinal assessment will provide additional evidence regarding the effectiveness of these instructional approaches and their impact on student learning.

8.6 Other Activities and Ideas

8.6.1 Unplugged Activity-More Processors (MP) Writing Activity

Problem Description, Prerequisites, and Learning Outcomes

The MP Writing Exercise is an unplugged classroom activity that introduces students to fundamental parallel computing concepts through a collaborative, hands-on simulation. The activity requires only paper, pencils or pens, and a timer, making it easy to implement in most classroom settings. Students assume the role of processors and complete a series of writing tasks, with execution times recorded. As the activity progresses, students experience how different execution models affect performance and develop an intuitive understanding of sequential execution, concurrency, shared-memory parallelism, scheduling, load balancing, speedup, scalability, and the limitations described by Amdahl's Law.

Prerequisites

There are no prerequisites for this activity, making it suitable for students with little or no prior exposure to programming or PDC concepts.

Learning Outcomes

Upon completion of the activity, students should be able to:

- Explain the differences between sequential execution and parallel execution.
- Explain how partitioning data across multiple processors supports parallel processing.
- Describe how distributing work among processors contributes to effective load balancing.
- Identify the communication and synchronization overhead that can occur during parallel execution.
- Explain why the processor that finishes last determines the overall execution time of a parallel program.
- Explain how the activity demonstrates the concepts of speedup, scalability, and Amdahl's Law.

Implementation Details

Table 8.12 summarizes the estimated time required for each component of the activity, from the introduction and setup through execution, discussion, and optional assessments. These estimates serve as a general guideline and may vary depending on class size, student engagement, and the amount of discussion incorporated.

Table 8.12: Estimated Instructional Time for Unplugged MP Writing Activity

Component	**Time (minutes)**
Pre-survey	3–5
Introduction and Instructions	5–10
Activity Setup	3–5
Activity Execution	20–25
Discussion and Debrief	5–10
Post-survey/Engagement Survey	3–5
Total Time	**45–65**

The activity is organized into five phases, each building upon the previous one.

In the first phase, a single student acts as the processor and completes two tasks sequentially: writing the sentence "More Processors Are Not Always The Best" and then assigning an index number to every character in the sentence. A second student records the total execution time. This phase establishes the baseline for sequential execution comparison.

In the second phase, the same student again acts as a single processor but now performs both tasks simultaneously by alternating between writing one character of the sentence and

immediately recording its corresponding index. This interleaved execution demonstrates multitasking on a single processor and allows students to compare the performance and cognitive demands of multitasking versus purely sequential execution.

The third phase introduces parallel execution using two processors. One student writes the sentence while a second student simultaneously assigns character indices. A third student records the completion times, with the overall execution time determined by the slower of the two participants. Students compare these results with those from previous phases and discuss how parallel execution can reduce completion time while introducing synchronization considerations.

The fourth phase expands the activity to multiple processors by dividing both the sentence and indexing tasks among four participants. Two students independently write different portions of the sentence while a third student indexes the first portion. After completing the first section, the third student communicates the next available index value to a fourth participant, who then indexes the second portion of the sentence. A fifth participant records the execution times. This phase introduces concepts such as task decomposition, workload distribution, inter-process communication, synchronization, and load balancing, illustrating that parallel tasks often depend on one another and require coordination to complete successfully.

An optional fifth phase demonstrates the limitations of Amdahl's Law and introduces the concept of weak scaling. Rather than simply adding more processors to the existing workload, an additional task, counting the number of characters in each word is introduced, along with an increase in the number of participants. Students observe that increasing both the workload and the number of processors can improve resource utilization, while also recognizing that some portions of a problem remain sequential and ultimately limit the achievable speedup.

After each phase, students compare execution times and discuss how the execution model affected performance. Reflection questions encourage students to consider why some phases were faster than others, identify sources of communication and synchronization overhead, evaluate how effectively the workload was balanced among participants, and determine which portions of the tasks could or could not be parallelized. The activity concludes with a class discussion connecting students' observations to formal parallel computing concepts, including speedup, scheduling, concurrency, scalability, load balancing, communication overhead, synchronization, and Amdahl's Law. Because students physically perform the tasks and observe the timing differences firsthand, the exercise provides an intuitive bridge between theoretical concepts and practical parallel-programming design, making it an effective introductory activity before implementing similar algorithms in code.

- **Materials Required** Each group will need blank paper and writing utensils (pens or pencils) to complete the writing and indexing tasks. A timing device, such as a smartphone timer, stopwatch, or other digital timer, is also required to measure the execution time for each phase of the activity.

  Instructional Material (Lectures, Videos, Assignments, Tests) `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter8-HPU`

Experience and Teaching Tips

The activity requires only paper, writing utensils, and a timer, making it a low-cost and easily deployable exercise that can be incorporated into nearly any classroom setting with minimal preparation. Because no specialized hardware or software is needed, instructors can easily incorporate the exercise into a standard class period with minimal preparation. Experience teaching this activity: provide clear instructions before the activity begins, assign specific roles (e.g., processors and timer), and allow students to rotate through different roles to foster greater engagement and a deeper understanding of the underlying concepts. It is also helpful to conclude the activity with a guided discussion that encourages students to reflect on how communication overhead, synchronization, and task dependencies affected performance and to connect these observations to parallel computing principles such as scalability and Amdahl's Law.

8.6.2 Ideas on Activities

The Flag Maker activity is simple to implement but requires advance preparation and coordination. Instructors must prepare gridded flag templates, colored markers or crayons, and timers for each group. Students may also need time to understand their assigned processor roles and coordinate their work, particularly during the parallel phases where they share the same worksheet and drawing tools. These shared resources intentionally create contention and synchronization challenges, providing natural opportunities to discuss mutual exclusion, scheduling, and communication in parallel systems. Several alternatives can simplify implementation while maintaining the learning objectives. Instructors may substitute the Mauritius flag for another gridded image, pixel art, or a simple geometric design; use colored pencils, stickers, or other coloring materials; or adapt the activity for online or hybrid courses using collaborative drawing tools. The activity can also be scaled by adjusting the grid size, number of colors, or number of processors to accommodate different class sizes and instructional goals.

The primary logistical challenge for the Penny Sorting activity is obtaining, transporting, and organizing enough pennies for the class. Depending on class size, several hundred or even thousands of pennies may be required. Instructors should plan ahead to acquire sufficient pennies and store them in labeled bags that are already divided for each phase of the activity. As an alternative, laminated or paper-cut pennies with printed years can be used to reduce cost, weight, and preparation time while still preserving the activity's learning objectives. These cutouts are easier to transport, organize, and redistribute between phases, making them especially useful for large classes or conference workshops.

The MP Writing Exercise is an easy-to-implement unplugged activity that requires only paper, pencils or pens, printed instructions, and a timer. Students assume the roles of processors and complete a series of writing and indexing tasks using sequential, multitasking, and parallel execution models while recording execution times for comparison. As the activity progresses, students experience concepts such as concurrency, synchronization, scheduling, communication overhead, load balancing, speedup, scalability, and Amdahl's Law. Because participants must coordinate their work and communicate intermediate results during the parallel phases, the activity naturally demonstrates the challenges and trade-offs associated

with parallel computing. Following each phase, students compare execution times and discuss how the different execution models affected overall performance.

The MP activity is flexible and can be adapted for a variety of classroom settings. Instructors may substitute the original sentence with another sentence, code fragment, or mathematical expression, add additional tasks to illustrate concepts such as task decomposition or weak scaling, or adjust the number of participants to accommodate different class sizes. The activity can also be adapted for online or hybrid instruction using collaborative editing tools. To maximize student learning, instructors should encourage students to predict the performance of each execution model before beginning a phase and then compare those predictions with the recorded results. These discussions help students connect the hands-on experience to fundamental parallel computing principles and reinforce the relationship between theoretical concepts and practical execution.

8.7 Conclusion

Introducing PDC concepts early in the computer science curriculum helps students develop a strong conceptual foundation for topics they will encounter throughout their academic and professional careers. As described in this chapter, HPU incorporated several unplugged activities and one plugged activity into an introductory CS1 course to introduce core PDC concepts without requiring prior experience in parallel programming. Through hands-on, collaborative learning experiences, students explored concepts such as concurrency, message passing, shared memory, scheduling, load balancing, speedup, and Amdahl's Law in an engaging and accessible way. The unplugged activities made "parallel thinking" approachable by allowing students to physically simulate parallel processes, while the plugged activity reinforced these concepts by demonstrating their application in a computational setting. The unplugged activities primarily addressed the Remembering and Understanding levels of Bloom's Revised Taxonomy, whereas the plugged activity provided opportunities for students to Apply these concepts in a computational setting. Collectively, these activities established a conceptual foundation upon which students can later develop higher-order skills associated with Analyzing, Evaluating, and Creating as they progress through more advanced PDC coursework.

Several lessons emerged from this work. First, introducing PDC concepts through active, collaborative learning can make topics that are often perceived as abstract or advanced more accessible to beginning programmers. Students responded positively to the unplugged activities, which encouraged discussion, teamwork, and problem-solving before introducing implementation details. Second, combining unplugged and plugged activities provided a natural progression from conceptual understanding to computational application, helping students connect theoretical ideas with programming practice. Finally, embedding PDC concepts within existing CS1 topics demonstrated that meaningful PDC instruction can be integrated into an introductory programming course without requiring a separate course or substantial curricular changes.

Based on these experiences, several teaching recommendations may benefit instructors interested in incorporating PDC into early computing courses. Instructors should clearly connect each activity to familiar CS1 concepts, such as loops, arrays, or methods, so students

recognize the relevance of PDC within the broader curriculum. Sufficient time should be allocated for discussion and debriefing after each activity, as these conversations often help students relate their experiences to formal PDC terminology. Small-group collaboration should be encouraged to promote peer learning, while instructors should actively facilitate discussions and provide guidance as needed. Finally, unplugged activities should be paired with a simple programming exercise whenever possible to reinforce conceptual understanding through practical application.

Although the results are encouraging, several limitations should be considered. The study was conducted at a single institution with relatively small class sizes, limiting the generalizability of the findings. Much of the evaluation relied on student perceptions, self-reported surveys, and short learning assessments rather than long-term measures of knowledge retention or programming performance. In addition, the activities were implemented within one CS1 course; therefore, the findings may not directly transfer to institutions with different student populations, programming languages, or curricular structures.

Future work will focus on expanding both the implementation and evaluation of these instructional activities. Additional unplugged and plugged-in activities will be developed to introduce a broader range of foundational PDC concepts and provide students with increasingly rich opportunities to connect conceptual understanding with computational practice. Multi-institutional studies involving larger and more diverse student populations, together with longitudinal data collection, will provide stronger evidence regarding the effectiveness of integrating PDC concepts early in the computing curriculum. Future research will also examine how early exposure to PDC influences student performance and preparedness in subsequent computer science courses, particularly upper-level PDC, systems, software engineering, and other courses where concurrency and parallelism play an important role. Continued refinement of the instructional materials based on student input and classroom experiences will further improve their effectiveness and usability. Finally, ongoing assessment using validated learning instruments, performance-based measures, and long-term follow-up studies will provide deeper insight into how early PDC instruction influences students' conceptual understanding, programming ability, problem-solving skills, and long-term retention of fundamental parallel and distributed computing concepts.

Bibliography

[1] Lorin W. Anderson and David R. Krathwohl, editors. A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives. Longman, New York, NY, 2001.

[2] Beth L. Wiggins, Sarah L. Eddy, Lara Wener-Fligner, Karin Freisem, Daniel Z. Grunspan, Elli J. Theobald, Janice Timbrook, and Alison J. Crowe. Aspect: A survey to assess student perspective of engagement in an active-learning classroom. CBE—Life Sciences Education, 16(2):ar32, 2017.

Appendix

Github Link to Instructional Material

https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter8-HPU

Detailed Pre and Post Syllabus

Organization of Material

The GitHub repository is organized into the following directories and files, each containing resources that support the implementation, evaluation, and replication of the PDC activities.

```
Chapter8-HPU/
  Detailed_pre_and_Post_Syllabus/
  Evaluation_Data_and_Analysis/
  Plugged-In_Activities/
  Unplugged_Activities/
  readme.md
```

Detailed_pre_and_Post_Syllabus

Purpose: This directory contains the course syllabi used before and after the integration of the Parallel and Distributed Computing (PDC) activities.

Contents: Pre- and post-intervention course syllabi, including course descriptions, learning outcomes, prerequisites, grading policies, and schedules.

Evaluation_Data_and_Analysis

Purpose: This directory contains the evaluation data and analysis materials used to assess the impact of the Parallel and Distributed Computing (PDC) activities.

Contents: Assessment data, survey responses, evaluation metrics, statistical analyses, and supporting documentation related to student learning outcomes and course effectiveness.

Plugged-In_Activities

Purpose: This directory contains the plugged-in programming activities used to introduce and reinforce PDC concepts.

Contents: Programming exercises, source code, lab activities, supporting documentation, and instructional resources for the plugged-in PDC activities.

Unplugged_Activities

Purpose: This directory contains the unplugged activities used to introduce foundational PDC concepts without requiring programming.

Contents: Activity guides, worksheets, instructional materials, facilitator resources, and supporting documentation for the unplugged PDC activities.

readme.md

Purpose: This file provides an overview of the contents and organization of the X directory.

Contents: A description of the directory, its resources, navigation information, and guidance for using the materials.

Survey and Other Anonymized Data and Analysis

On the GitHub link (`https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter8-HPU`), the directory "Evaluation_Data_and_Analysis" contains the de-identified and anonymized data and the analytical materials used for the evaluation of the PDC interventions.